Python 模块 requests 基本用法

简介

Python 的 requests 模块是一个流行的第三方库，用于发送 HTTP 请求。它提供了一组简洁且易于使用的 API，使得在 Python 中进行网络通信变得更加简单和灵活。

---

1. 基本概念

1.1 HTTP 协议

HTTP协议（Hypertext Transfer Protocol）是一种用于在网络上发送和接收HTML页面或其他内容的协议，是基于客户端-服务器模型，客户端发起HTTP请求，服务器响应请求并返回相应的内容。HTTP协议使用URL（Uniform Resource Locator）来标识互联网上的资源，如网页、图片、视频等。客户端通过发送HTTP请求来获取特定URL对应的资源，请求包括请求方法（如GET、POST）、头部信息（如浏览器类型、请求资源类型等）和可选的请求体（如表单数据、请求参数等）。

主要作用：

1. **传输数据**：在客户端和服务器之间传输各种类型的数据。
2. **客户端和服务器通信**：定义通信规则，客户端发送请求，服务器响应。
3. **资源定位和标识**：使用 URL 定位和标识互联网上的资源。
4. **无状态协议**：每个 HTTP 请求都是独立的，服务器不保留状态信息。
5. **安全性**：HTTP 是明文传输，可使用 HTTPS 添加 SSL/TLS 加密层保障安全。

HTTP协议是在互联网上进行数据传输和通信的基础协议，它为客户端和服务器之间提供了一种简单、开放的通信方式，使得互联网上的信息交流和资源访问变得更加便捷和高效。其主要作用是

> 传输数据：HTTP协议是一种用于传输数据的协议，它可以在客户端和服务器之间传输各种类型的数据，如HTML页面、图片、视频、音频等。通过HTTP，用户可以在浏览器中访问网页，下载文件，发送和接收数据等。
>
> 客户端和服务器通信：HTTP协议定义了客户端和服务器之间的通信规则。客户端发送HTTP请求到服务器，服务器解析请求并发送HTTP响应回客户端。这种通信模式使得客户端可以请求特定资源，服务器可以根据请求进行处理并返回相应的内容。
>
> 资源定位和标识：HTTP协议使用URL来定位和标识互联网上的资源。URL由协议类型、服务器地址、资源路径等组成，通过URL可以唯一确定一个资源的位置。
>
> 无状态协议：HTTP协议是无状态的，即服务器不会保留与客户端之间的任何状态信息。每个HTTP请求都是独立的，服务器只处理请求并返回相应的响应，不会对之前的请求进行记忆。这种无状态的设计使得HTTP协议具有简单、可扩展的特性。
>
> 安全性：HTTP协议本身是明文传输的，数据在传输过程中可能会被窃听或篡改。为了保障通信的安全性，可以使用HTTPS（HTTP Secure）协议，它在HTTP协议之上添加了SSL/TLS加密层，对数据进行加密和认证。

1.2 GET 请求

GET请求是HTTP协议中的一种请求方法，用于从服务器获取资源。它是最常见和最简单的请求方法之一。
GET请求中的请求参数会暴露在URL中，因此，不应将敏感信息通过GET请求传递。对于传输敏感信息（例如：用户、密码）或需要修改服务器上资源状态的请求，应使用POST请求或其他更安全的请求方法。

特点：

- 请求参数通过URL中的查询字符串传递，如：http://www.baidu.com/path?param1=value1¶m2=value2
- GET请求对请求参数的长度有限制，因为URL的长度是有限制的，不适合传输大量的数据。
- GET请求是幂等的，即多次重复发送同样的GET请求，不会对服务器产生副作用，也不会修改服务器上的资源状态。
- GET请求可以被缓存，如果相同的GET请求被发送多次，中间的请求可以通过缓存直接返回响应结果。

优点：

- 简单和直观：GET请求是HTTP协议中最简单的请求方法，只需要通过URL进行请求信息的传递，易于理解和实现。
- 可缓存：GET请求可以被缓存，如果相同的GET请求被重复发送，中间的请求可以通过缓存直接返回响应结果，减少了服务器的负载和网络传输时间。
- 可书签化：由于GET请求的参数附加在URL中，因此可以将带有GET请求的URL添加到书签中，方便用户保存和分享特定的资源。
- 可见性：GET请求的参数和请求信息都会暴露在URL中，可以直观地看到请求的内容，方便调试和问题排查。

缺点：

- 长度限制：GET请求的请求参数通过URL的查询字符串传递，URL的长度是有限制的，所以对于大量数据的传输不合适，存在长度限制问题。
- 安全性问题：GET请求中的请求参数暴露在URL中，可能被其他人在网络传输过程中截获或记录，存在安全风险。敏感信息（如密码）不应通过GET请求传递。
- 副作用问题：GET请求属于幂等请求，即多次发送同样的GET请求不会对服务器产生副作用。但对于涉及修改服务器资源状态或产生其他副作用的请求（如增、删、改等操作），不应该使用GET请求。

1.3 POST 请求

- POST请求是HTTP协议中的一种请求方法，用于向服务器提交数据，并请求服务器处理该数据。与GET请求相比，POST请求常用于发送较大量的数据、修改服务器上的资源状态以及进行敏感操作等。
- POST请求不仅可以传输表单数据，也可以传输其他类型的数据，如JSON、XML等。使用POST请求时，需要根据具体的场景和需求，将数据格式化和编码成对应的请求体数据类型。

特点：

- 数据传输较安全：相对于GET请求将数据暴露在URL中，POST请求将数据放在消息体中发送，对于敏感信息更为安全。
- 请求参数长度无限制：POST请求中的参数和数据不会受到URL长度限制，能够传输较大量的数据。
- 不可缓存：由于POST请求需要向服务器提交数据，每次请求的数据都可能不同，所以大部分情况下不能被缓存，需要实时重新请求。
- 非幂等性：POST请求可能对服务器资源产生副作用，即每次发送相同的POST请求，可能会对服务器资源状态进行修改。

优点：

- 数据安全性更高：相对于GET请求，POST请求将数据放在请求的消息体中传输，而不是直接附加在URL中，因此对于敏感信息的传输更加安全，减少了数据被恶意拦截或篡改的风险。
- 容量大小无限制：POST请求传输的数据没有URL长度限制，可以传输较大的数据量，适合上传文件或传输大量文本数据。
- 更好的请求语义：POST请求用于向服务器提交数据并请求处理，适合于对服务器资源进行修改、创建等操作，具有更好的请求语义。使用POST请求可以清晰地表达意图，使服务器能够更好地处理和响应请求。
- 支持多种数据格式：POST请求可以使用不同的Content-Type来指定数据格式，可以传输多种类型的数据，如表单数据、JSON数据、XML数据等，具有更大的灵活性。

缺点：

- 非幂等性：POST请求可能对服务器资源产生副作用，即每次发送相同的POST请求，都可能会对服务器资源状态进行修改，而不仅仅是获取资源。这意味着相同的POST请求不能重复执行，可能会导致不可预料的结果。
- 不可缓存：由于POST请求中往往包含了对服务器资源的修改，为了保证数据的一致性，POST请求通常不能被缓存，需要实时重新请求，增加了服务器的负载和网络传输时间。
- 请求复杂度相对较高：相对于GET请求，POST请求在构造和发送上相对复杂一些，需要将数据放入请求的消息体中，并设置合适的Content-Type、Content-Length等请求头部信息。

1.4 GET 和 POST 的区别

- GET请求适用于获取资源、查询或浏览数据等操作，传输的数据量较小且不涉及敏感信息。
- POST请求适用于向服务器提交数据、修改资源状态等操作，支持传输较大量的数据和敏感信息的安全传输。

参数位置和传输方式

- GET请求：参数通过URL的查询字符串传递，附加在URL的末尾，以?开头，多个参数之间使用&分隔，例如：http://www.baidu.com/path?param1=value1¶m2=value2。数据可以通过URL进行传输，也可以通过请求头的字段进行传输。
- POST请求：参数和数据放在请求的消息体中，通过请求头部的Content-Type字段来指定数据的类型，常见的有application/x-www-form-urlencoded、multipart/form-data、application/json等。

数据传输安全性

- GET请求：参数和数据暴露在URL中，容易被拦截、篡改或记录，不适合传输敏感信息。
- POST请求：参数和数据放在请求的消息体中传输，对于敏感信息的安全性更高。

请求语义和副作用

- GET请求：主要用于从服务器获取资源，获取数据的语义明确，是幂等的，多次请求结果相同，不会对服务器产生副作用。
- POST请求：主要用于向服务器提交数据，请求可能对服务器产生副作用，常用于修改、创建资源等操作，不具备幂等性。

URL长度限制

- GET请求：由于参数在URL中传递，URL的长度存在限制（不同浏览器和服务器对URL长度的限制不同），不适合传输大量数据。
- POST请求：没有URL长度限制，可以传输较大量的数据。

缓存

- GET请求：可以被缓存，如果相同的GET请求被重复发送，中间的请求可以通过缓存直接返回响应结果，减少了服务器的负载和网络传输时间。
- POST请求：通常不会被缓存，因为POST请求可能对服务器资源产生副作用，需要实时重新请求。

对比项	GET	POST
参数位置和传输方式	参数在 URL 中	参数在消息体中
数据传输安全性	暴露参数	更安全
请求语义和副作用	用于获取资源（幂等）	用于提交/修改数据（非幂等）
URL 长度限制	有限制	无限制
缓存	可缓存	通常不缓存

---

2. 发送请求

2.1 发送 GET 请求

语法：

```
requests.get(
    url             #必选参数，指定URL地址
    params=None     #可选参数，用于指定查询参数，可以是一个字典或一个字符串。
    **kwargs        #可选参数，用于传递其他参数，例如请求头、超时时间等。
)

**kwargs 可以是：
    headers：设置请求头信息
    timeout：设置请求的超时时间(单位：秒)
```

示例：

```
import requests

# 发送get请求
response = requests.get('https://www.baidu.com/')

# 打印状态码
print(response)

# 打印内容
print(response.text)
```

结果显示:

```
<Response [200]> 状态码
<!DOCTYPE html>  内容
<!--STATUS OK--><html> <head><meta http-equiv=content-t
                </script> <a href=//www.baidu.com/more/
```

CSDN @〖羊头〗Isy

1.发送带查询参数的 get 请求

```
params = {'key1': 'value1', 'key2': 'value2'}
response = requests.get('https://www.baidu.com/', params=params)

'''

params = {'key1': 'value1', 'key2': 'value2'}:
    这里定义了一个名为 params 的 Python 字典。
    在发送 HTTP 请求时，这个字典会被用作查询字符串（Query String）参数。
    它将被转换为 ?key1=value1&key2=value2 这样的格式，并附加到请求的 URL 后面。

response = requests.get('https://www.baidu.com/', params=params):
    requests.get(...): 调用 requests 库的 get 方法，表示要发起一个 HTTP GET 请求。
    'https://www.baidu.com/': 这是你要请求的目标 URL 地址。
    params=params: 将上面定义的 params 字典作为 params 参数传递给 get 方法。requests 库会自动处理这个字典，将其格式化并附加到 URL 上。

'''
```

2.发送GET请求并加入其他参数（设置请求头和请求超时时间）

```
headers = {'User-Agent': 'Mozilla/5.0'}
response = requests.get('https://www.baidu.com/', headers=headers, timeout=5)

'''

headers = {'User-Agent': 'Mozilla/5.0'}:
    这里定义了一个名为 headers 的 Python 字典。
    这个字典代表了 HTTP 请求头（Request Headers）。请求头包含了关于请求的元信息。
    'User-Agent': 'Mozilla/5.0' 是一个常见的请求头，它告诉服务器发起请求的是什么类型的客户端（通常是浏览器）。通过设置这个值，可以让我们的脚本伪装成一个普通的网页浏览器，而不是一个自动化程序，这有助于绕过一些网站的反爬虫机制。

response = requests.get('https://www.baidu.com/', headers=headers, timeout=5):
    requests.get(...): 发起一个 HTTP GET 请求。
    'https://www.baidu.com/': 目标 URL。
    headers=headers: 将上面定义的 headers 字典传递给 get 方法。这样，在发送请求时，就会包含我们指定的请求头信息。
    timeout=5: 设置请求的超时时间为 5 秒。这意味着如果服务器在 5 秒内没有响应，requests 库会主动中断请求并抛出一个 Timeout 异常。这是一个非常好的实践，可以防止你的程序因为等待一个无响应的服务器而无限期地挂起。

'''
```

特性	`params` (参数)	`headers` (请求头)
位置	位于 URL 的查询字符串部分，即 `?` 之后。	位于 HTTP 请求的头部区域。
内容	通常用于向服务器传递请求数据或过滤条件，例如搜索关键词、页码等。	用于传递关于请求本身的元信息，如客户端类型、允许的内容格式、身份认证信息等。
可见性	会直接显示在 URL 中，对用户和服务器都可见。	不会出现在 URL 中，对用户不可见，仅在 HTTP 协议层面传输。
数据格式	通常是键值对，会被编码后附加在 URL 后面，例如 `?key1=value1&key2=value2`。	也是键值对，但遵循 HTTP 头部的格式规范。

- 当你需要向服务器传递**业务逻辑相关的数据**（如搜索内容、ID 等），并且这些数据可以公开时，使用 `params`。
- 当你需要向服务器传递**协议或配置相关的信息**（如身份、数据格式偏好等），或者需要隐藏敏感信息时，使用 `headers`。

2.2 发送 POST 请求

语法：

```
requests.post(
    url             #必选参数，指定URL地址。
    data=None       #可选参数，传递表单数据。可以是一个字典、字符串或二进制数据。
    json=None       #可选参数，传递JSON数据。将自动将数据编码为JSON格式并设置请求头部的Content-Type为application/json。
    headers=None    #可选参数，指定HTTP请求的头部信息。可以通过字典形式传递。
    cookies=None    #可选参数，传递Cookie信息。可以是字典或CookieJar对象。
    auth=None       #可选参数，用于身份验证，可以是身份验证相关的对象，如HTTPBasicAuth或HTTPDigestAuth等。
    timeout=None    #可选参数，设置请求的超时时间(单位：秒)
    allow_redirects  #可选参数，用于设置是否允许重定向(默认True)。
    **kwargs        #可选参数，用于传递其他参数，例如proxies、verify（SSL验证）等。
)
```

示例：

```
import requests

# 发送post请求
response = requests.post('https://www.baidu.com/')

# 打印状态码
print(response)

# 打印内容
print(response.text)
```

```
<Response [302]> 状态码302表示临时重定向
<html>
<head>
<meta http-equiv="content-type" content="text/html;charset
<style data-for="result" id="css_result">
body{color:#333;background:#fff;padding:6px 0 0;margin:0;p
```

内容

CSDN @〖羊头〗lsy

```
import requests #用于发送 HTTP 请求的核心库。
import json # 用于处理 JSON 数据的内置库。
from requests.auth import HTTPBasicAuth #requests 库提供的一个类，用于实现 HTTP Basic Authentication（基本身份验证）。

# 简单请求
#发送一个最基础的 POST 请求到 https://www.baidu.com/。此请求没有任何额外的数据、头信息或参数。
response = requests.post('https://www.baidu.com/')

# 发送 JSON 数据
data = {'key': 'value'} #定义了一个 Python 字典
headers = {'Content-Type': 'application/json'} #准备了一个请求头，声明发送的数据是 JSON 格式。
response = requests.post('https://www.baidu.com/', json=data, headers=headers)

#发送带有请求头和超时时间的POST请求
url = 'https://www.baidu.com/'
headers = {'User-Agent': 'Mozilla/5.0'} #设置请求头，模拟一个浏览器访问。
response = requests.post(url , headers=headers, timeout=5) #设置请求的超时时间为 5 秒。

# 带身份验证
auth = HTTPBasicAuth('username', 'password') #创建一个认证对象，其中包含用户名和密码
response = requests.post('https://www.baidu.com/', auth=auth) #将认证对象传递给 auth 参数，requests 会在请求头中添加相应的认证信息。

# 带 Cookie
cookies = {'session_id': '123456'} #定义一个 Python 字典，代表要发送的 Cookie 键值对。
response = requests.post('https://www.baidu.com/', cookies=cookies) #将字典传递给 cookies 参数，requests 会将其格式化并添加到请求中。
```

运行示例：

```
import requests
import json
from requests.auth import HTTPBasicAuth

print("--- 示例 1: 简单请求 ---")
try:
    response = requests.post('https://www.baidu.com/')
    print(f"请求URL: {response.url}")
    print(f"响应状态码: {response.status_code}") # 预期: 200
    print(f"响应内容长度: {len(response.text)}") # 预期: 返回百度主页的HTML内容长度
    print("-" * 20)
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")
```

```python
print("\n--- 示例 2: 发送 JSON 数据 (修正版) ---")
data = {'key': 'value'}
try:
    # 正确的发送JSON的方式
    response = requests.post('https://www.baidu.com/', json=data)
    print(f"请求URL: {response.url}")
    print(f"请求头 (检查Content-Type): {response.request.headers.get('Content-Type')}") # 预期: application/json
    print(f"响应状态码: {response.status_code}") # 预期: 200
    print(f"响应内容长度: {len(response.text)}") # 预期: 返回百度主页的HTML内容长度
    print("-" * 20)
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")

print("\n--- 示例 3: 带请求头和超时 ---")
headers = {'User-Agent': 'Mozilla/5.0 (CustomApp v1.0)'}
try:
    response = requests.post('https://www.baidu.com/', headers=headers, timeout=5)
    print(f"请求URL: {response.url}")
    print(f"响应状态码: {response.status_code}") # 预期: 200
    print(f"响应内容长度: {len(response.text)}") # 预期: 返回百度主页的HTML内容长度
    print("-" * 20)
except requests.exceptions.Timeout:
    print("请求超时！")
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")

print("\n--- 示例 4: 带身份验证 ---")
try:
    # 百度主页不需要Basic Auth，所以会返回200，但验证信息会被忽略
    auth = HTTPBasicAuth('my_username', 'my_password')
    response = requests.post('https://www.baidu.com/', auth=auth)
    print(f"请求URL: {response.url}")
    print(f"响应状态码: {response.status_code}") # 预期: 200
    print(f"响应内容长度: {len(response.text)}") # 预期: 返回百度主页的HTML内容长度
    print("-" * 20)
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")

print("\n--- 示例 5: 带 Cookie ---")
cookies = {'custom_session': 'abc123', 'user_preference': 'dark_mode'}
try:
    response = requests.post('https://www.baidu.com/', cookies=cookies)
    print(f"请求URL: {response.url}")
    print(f"响应状态码: {response.status_code}") # 预期: 200
    print(f"响应内容长度: {len(response.text)}") # 预期: 返回百度主页的HTML内容长度
    print("-" * 20)
except requests.exceptions.RequestException as e:
    print(f"请求失败: {e}")
```

可能的输出结果:

```
--- 示例 1: 简单请求 ---
请求URL: https://www.baidu.com/
响应状态码: 200
响应内容长度: 2443
```

```
--------------------

--- 示例 2: 发送 JSON 数据 (修正版) ---
请求URL: https://www.baidu.com/
请求头 (检查Content-Type): application/json
响应状态码: 200
响应内容长度: 2443
--------------------

--- 示例 3: 带请求头和超时 ---
请求URL: https://www.baidu.com/
响应状态码: 200
响应内容长度: 2443
--------------------

--- 示例 4: 带身份验证 ---
请求URL: https://www.baidu.com/
响应状态码: 200
响应内容长度: 2443
--------------------

--- 示例 5: 带 Cookie ---
请求URL: https://www.baidu.com/
响应状态码: 200
响应内容长度: 2443
--------------------
```

2.3 发送 PUT 请求

语法:

```
requests.put(
    url             #必选参数，指定URL地址。
    data=None       #可选参数，传递表单数据。可以是一个字典、字符串或二进制数据。
    json=None       #可选参数，传递JSON数据。将自动将数据编码为JSON格式并设置请求头部的
Content-Type为application/json。
    headers=None    #可选参数，指定HTTP请求的头部信息。可以通过字典形式传递。
    cookies=None    #可选参数，传递Cookie信息。可以是字典或CookieJar对象。
    auth=None       #可选参数，用于身份验证，可以是身份验证相关的对象，如HTTPBasicAuth或
HTTPDigestAuth等。
    timeout=None    #可选参数，设置请求的超时时间(单位：秒)
    allow_redirects  #可选参数，用于设置是否允许重定向(默认True)。
    **kwargs        #可选参数，用于传递其他参数，例如proxies、verify（SSL验证）等。
)
```

示例:

```
# 发送put请求
response = requests.put('https://www.baidu.com/')

# 打印状态码
print(response)

# 打印内容
print(response.text)
```

运行结果：

```
<Response [302]> 状态码302表示临时重定向
<html>
<head>    内容
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<style data-for="result" id="css_result">
body{color:#333;background:#fff;padding:6px 0 0;margin:0;position:r
```

2.4 发送 DELETE 请求

语法：

```
requests.delete(
    url             #必选参数，指定URL地址。
    params=None     #可选参数，传递URL查询参数。可以是一个字典或字符串。
    headers=None    #可选参数，指定HTTP请求的头部信息。可以通过字典形式传递。
    cookies=None    #可选参数，传递Cookie信息。可以是字典或CookieJar对象。
    auth=None       #可选参数，用于身份验证，可以是身份验证相关的对象，如HTTPBasicAuth或
HTTPDigestAuth等。
    timeout=None    #可选参数，设置请求的超时时间(单位：秒)
    allow_redirects  #可选参数，用于设置是否允许重定向(默认True)。
    **kwargs        #可选参数，用于传递其他参数，例如proxies、verify（SSL验证）等。
)
```

示例：

```
# 发送delete请求
response = requests.delete('https://www.baidu.com/')

# 打印状态码
print(response)

# 打印内容
print(response.text)
```

运行结果

```
<Response [302]> 状态码302表示临时重定向
<html>
<head>    内容
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<style data-for="result" id="css_result">
body{color:#333;background:#fff;padding:6px 0 0;margin:0;position:r
```

3. 处理响应

3.1 获取响应内容

属性/方法	说明
`response.text`	以字符串形式返回
`response.content`	以字节形式返回
`response.json()`	以 JSON 解析返回（若非 JSON 格式则抛出异常）

```
response = requests.get('https://www.baidu.com/')
print(response.text)
print(response.content)
print(response.json())
```

以 "字符串" 形式返回响应内容

```
<!DOCTYPE html>
<!--STATUS OK--><html> <head><meta http-equiv=
                </script> <a href=//www.baidu.


Process finished with exit code 0
```

CSDN @〖羊头〗Isy

以 "字节" 形式返回响应内容

```
b'<!DOCTYPE html>\r\n<!--STATUS OK--><html> <he

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

以 "JSON" 形式返回响应内容（如果响应内容不是有效的JSON格式，则会抛出JSONDecodeError异常。）

```
    print(response.json())
  File "F:\software\Python\lib\site-packages\requests\models.py", li
    raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
requests.exceptions.JSONDecodeError: Expecting value: line 1 column
```

返回格式不是json

```
Process finished with exit code 1
```

CSDN @〖羊头〗Isy

3.2 获取响应状态

属性/方法	说明
`response.status_code`	返回状态码
`response.ok`	返回布尔值（2xx 为 True）
`response.raise_for_status()`	若状态码表明失败则抛出 HTTPError

```
response = requests.get('https://www.baidu.com/')
print(response.status_code)
print(response.ok)
print(response.raise_for_status())
```

获取响应的 "状态码"

```
200

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

获取响应的 "状态码的布尔值"

```
True
```

True表示正常

```
Process finished with exit code 0
```

CSDN @〖羊头〗Isy

"状态码"异常则抛出 "ConnectionError" 异常。

```
None
```

None说明状态码正常

```
Process finished with exit code 0
```

CSDN @〖羊头〗Isy

```
    r = adapter.send(request, **kwargs)
  File "F:\software\Python\lib\site-packages\requests\adapters.py"
    raise ConnectionError(e, request=request)
requests.exceptions.ConnectionError: HTTPSConnectionPool(host='www
```

状态码异常

```
Process finished with exit code 1
```

CSDN @〖羊头〗Isy

3.3 获取响应头部信息

属性/方法	说明
`response.headers`	返回头部字典
`response.headers['header_name']`	获取指定字段值

```
response = requests.get('https://www.baidu.com/'
print(response.headers)
print(response.headers['Date'])
```

以 "字典形式" 返回响应头部的全部信息

```
{'Cache-Control': 'private, no-cache, no-st

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

返回指定头部 "字段的值"

```
Sat, 22 Jul 2023 10:21:06 GMT

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

3.4 处理重定向

属性	说明
`response.history`	返回一个包含重定向历史的列表，每个元素是一个Response对象
`response.url`	返回最终响应的 URL

```
response = requests.get('https://www.baidu.com/')
print(response.history)
print(response.url)
```

返回一个包含 "重定向历史" 的列表

```
[]
    没有重定向
Process finished with exit code 0
```

CSDN @〖羊头〗Isy

返回最终响应的 "URL"

```
https://www.baidu.com/

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

3.5 获取其他响应信息

属性	说明
`response.encoding`	返回响应使用的字符编码。
`response.cookies`	返回响应的Cookies信息，可以进行进一步操作

```
response = requests.get('https://www.baidu.com/')
print(response.encoding)
print(response.cookies)
```

返回响应的 "字符编码"

```
ISO-8859-1

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

返回响应的 "cookies信息"

```
<RequestsCookieJar[<Cookie BDORZ=27315 for .baidu.com/>]>

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

3.6 指定内容字符集

```python
# 获取结果
response = requests.get('https://www.baidu.com/')

# 按内容展示
html = response.text

# 指定内容的字符集
html = response.content.decode('utf-8')

# 输出结果
print(html)
```

---

4. 会话管理和 Cookie

4.1 会话管理

- 发送多个相关请求时，使用会话（Session）对象来管理这些请求，以便在多个请求之间共享状态、Cookie和身份验证等信息。
- 创建一个会话对象后，身份信息和cookie都可以共享，适用于多个相关联的请求。

> 如果说 **Cookie 是“通行证”**，那么 **会话就是“一次性储物柜”或者“专属通道”**。
>
> 在 `requests` 库中，`requests.Session()` 的核心作用是**“状态保持”**和**“性能优化”**。
>
> 以下是会话的 4 大核心作用：
>
> **1. 自动管理 Cookie（最常用、最重要）**
>
> 这是会话最核心的功能。如果没有会话，你需要手动处理每一个 Cookie。
>
> - **不用会话：**
> 1. 你登录了，服务器返回 `Set-Cookie`。
> 2. 你必须手动提取这个 Cookie。
> 3. 访问下一页时，你必须手动把 Cookie 塞进请求头里。
> 4. **一旦忘记携带，就会掉线。**
> - **使用会话：**
> 1. 你使用 `session.post()` 登录。
> 2. **Session 自动保存**服务器返回的 Cookie。
> 3. 后续使用 `session.get()` 访问其他页面，**Session 自动带上**刚才保存的 Cookie。
> 4. **全程自动化，无需手动干预。**
>
> **2. 保持连接池（性能优化）**
>
> 这属于底层的网络优化，但对爬虫速度影响很大。
>
> - **不用会话：** 每次请求 `requests.get()` 都会建立一个新的 TCP 连接（三次握手），请求完就断开。如果爬取 1000 个页面，就要握手 1000 次。
> - 使用会话：
> Session 底层维护了一个连接池
> - 第一次请求建立连接后，连接**不会立即断开**，而是保持在池子里。
> - 第二次请求时，直接复用刚才的连接通道。
> - **结果：** 爬虫运行速度会显著提升，尤其是在高并发请求时。

3. 统一配置全局参数

如果你需要对所有请求设置统一的 Header（请求头）或代理，Session 非常方便。

- **场景：** 网站检查 `User-Agent`，你所有请求都必须带这个头。
- **作用：** 只需要给 Session 对象设置一次，后续所有通过该 Session 发出的请求都会自动带上。

4. 实现多账号隔离

- **场景：** 你有两个账号，账号 A 要爬取数据，账号 B 也要爬取数据。
- 作用：

  创建两个不同的 Session 对象。

  - Session A 里的 Cookie 属于账号 A。
  - Session B 里的 Cookie 属于账号 B。
  - 它们互不干扰，就像开了两个不同的浏览器窗口。

```
import requests

# 创建会话对象
session = requests.Session()

# 设置会话级别的头部信息(可选)
session.headers.update({'User-Agent': 'Mozilla/5.0'})

# 第一个请求(使用post请求登录)
response1 = session.post('http://xxx.com/login', data=login_data)
#后续请求自动携带 Cookie
# 第二个请求(登录以后的其他操作1)
response2 = session.get('http://xxx.com/xxx1')

# 第三个请求(登录以后的其他操作2)
response3 = session.get('http://xxx.com/xxx2')

# 关闭会话
session.close()
```

使用会话对象的优点

1. 自动管理Cookie：会话对象会自动在请求中发送和更新Cookie信息，无需手动处理。
2. 共享会话级别信息：会话对象可用于共享头部信息、身份验证等。这避免了在每个请求中单独设置这些信息的麻烦。
3. 连接重用：会话对象会自动重用连接，提高请求性能。

4.2 Cookie 操作

在爬虫开发、Web自动化测试或网络编程中，获取 Cookie 的作用至关重要。简单来说，**Cookie 是服务器用来识别用户身份的“通行证”或“钥匙”。**

获取 Cookie 主要有以下 5 大核心作用：

1. 维持登录状态（会话保持）—— 最核心的作用

这是最常见的需求。HTTP 协议本身是**无状态**的（服务器记不住你是谁）。

- **场景：** 你用代码模拟登录了某个网站（比如淘宝、GitHub）。

- **问题：** 如果你不保存登录后服务器返回的 Cookie，当你下一次请求“个人中心”页面时，服务器会认为你是新访客，把你踢回登录页。
- **作用：** 获取 Cookie 并在后续请求中携带它，服务器就会认为你一直是“登录状态”，从而允许你访问受保护的页面（如订单页、购物车）。

2. 绕过验证码（降低开发难度）

验证码是爬虫的噩梦。

- **场景：** 某网站登录需要复杂的滑块验证码或点选验证码。
- 策略：
  1. 你可以先手动在浏览器上登录，成功后从浏览器中**复制 Cookie**。
  2. 将这个 Cookie **填充**到你的爬虫代码中（`requests` 或 `Session`）。
  3. **作用：** 爬虫直接拥有了“已登录”的身份，完全跳过了登录和验证码的环节，省时省力。

3. 反爬虫与身份伪装

很多网站通过检查 Cookie 来判断访问者是不是机器人。

- 作用：
  - **模拟真实用户：** 浏览器访问时通常会携带正常的 Cookie。如果你的请求不带 Cookie，服务器可能会拒绝访问（返回 403 Forbidden）。
  - **User-Agent 配合：** 只有 Header 没有 Cookie，很容易被识别为脚本。获取并携带 Cookie 可以让爬虫看起来更像一个真实的浏览器用户。

4. 分析与逆向工程

在开发爬虫前，我们需要分析目标网站的逻辑。

- 作用：
  - 通过观察 `response.cookies`，我们可以知道网站使用了哪种会话技术（如 PHPSESSID, JSESSIONID, AWSALB 等）。
  - 分析 Cookie 的生成逻辑：有些网站的 Cookie 是通过 JavaScript 动态生成的（如 `acw_sc__v2`）。通过获取并对比 Cookie，我们可以判断是否需要去破解 JS 代码才能生成合法的 Cookie。

5. 实现多账号/多会话管理

- **场景：** 你需要同时爬取 10 个不同账号的数据。
- **作用：** 每个账号对应一个独立的 Cookie 文件或对象。获取并存储这些 Cookie，可以让爬虫在同一台机器上区分不同用户的身份，互不干扰（类似于浏览器的“多开窗口”功能）。

1. 添加 Cookie

```python
import requests

url = 'http://xxx.com'
cookies = {'name': 'value'}

# 请求中添加cookie值
response = requests.get(url, cookies=cookies)
```

2. 获取 Cookie值

```python
import requests

# 1. 发送请求
# 访问百度首页，服务器通常会在响应头中返回 Set-Cookie 字段
response = requests.get('https://www.baidu.com/')

# 2. 直接打印 Cookie 全部信息
# response.cookies 是一个 RequestsCookieJar 对象
# 打印它会显示类似 <RequestsCookieJar[<Cookie BDORZ=27315 for .baidu.com/>]> 的格式
# 这能让你快速查看服务器返回了哪些 Cookie
print(response.cookies)

# 3. 通过遍历获取具体的名称和值
# RequestsCookieJar 对象是可迭代的，可以像列表一样遍历
for cookie in response.cookies:
    # 每一个 cookie 对象都包含多个属性：
    # cookie.name  : Cookie 的名称 (如 'BDORZ')
    # cookie.value : Cookie 的值  (如 '27315')
    # cookie.domain: Cookie 所属的域名
    print(cookie.name, cookie.value)
```

```
<RequestsCookieJar[<Cookie BDORZ=27315 for .baidu.com/>]>
BDORZ 27315

Process finished with exit code 0
```

CSDN @〖羊头〗Isy

3. 删除 Cookie

```python
import requests

# 1. 发送请求
# 当我们访问百度时，服务器会在响应头中返回一些 Cookie（通常是用于识别用户会话、追踪行为等）
response = requests.get('https://www.baidu.com/')

# 2. 删除名称为 'BDORZ' 的 Cookie
# 这里的 response.cookies 是服务器发给我们的 Cookie 集合
# 假设服务器返回了一个名为 'BDORZ' 的 Cookie，这行代码会把它从集合中移除
response.cookies.pop('BDORZ')

# 3. 删除所有 Cookie
# 清空当前响应中携带的所有 Cookie 信息
response.cookies.clear()
```

4. 会话中添加 Cookie

```python
import requests

# 1. 创建一个会话
# Session 对象能够跨请求保持某些参数（比如 Cookie）
# 它就像打开了一个独立的浏览器窗口
session = requests.Session()

# 2. 添加 Cookie 到会话
# 这里的操作相当于在浏览器中手动设置了 Cookie，或者是登录成功后服务器返回了 Cookie
# 格式：字典形式 {'cookie名': 'cookie值'}
```

```
cookies = {'name': 'value'}

# 使用 update 方法将 Cookie 写入会话的 cookie jar 中
# 写入后，这个 Session 后续发出的所有请求都会自动携带这个 Cookie
session.cookies.update(cookies)

# 3. 使用包含 Cookie 的会话发送请求
# 此时发送的请求，HTTP Header 里会自动带上 Cookie: name=value
# 这通常用于访问需要登录权限的页面
response = session.get(url)

# 4. 清除所有 Cookie
# 这相当于点击了浏览器的“清除缓存”或“退出登录”按钮
# 清除后，Session 恢复到“未登录”状态，后续请求将不再携带之前的 Cookie
session.cookies.clear()
```

---

5. 异常处理

5.1 捕获所有异常

- RequestException 是所有异常的基类，可以用于捕获所有的异常情况。当发生网络请求错误时，例如连接超时、DNS解析失败等，可以使用该异常来捕获并处理。
- **requests.exceptions.RequestException**

```
import requests
# RequestException 是所有 requests 异常的祖宗类
# 它包含了：ConnectionError, Timeout, HTTPError, TooManyRedirects 等
from requests.exceptions import RequestException

try:
    response = requests.get(url)
    print('响应正常，执行下一步')

# ============================================
# RequestException “万能捕获”机制详解
# ============================================
# 无论发生以下哪种具体的错误，都会被这个 except 捕获：
#
# 1. 网络连接问题 -> 触发错误，但因为它是 RequestException 的子类，所以会被捕获。
# 2. 请求超时 -> 也会被捕获。
# 3. HTTP 状态码错误 (如果代码里加了 raise_for_status) -> 也会被捕获。
# 4. URL 格式错误、重定向过多等 -> 都会被捕获。
#
# 好处：代码简短，不会漏掉任何 requests 相关的错误，保证程序绝对不会因为网络问题而崩溃。
# 坏处：无法针对具体的错误类型（比如是断网了还是超时了）做精细化的处理。
except RequestException as e:
    print("请求发生异常:", str(e))
```

5.2 捕获状态码异常

- HTTPError 表示HTTP错误状态码（4xx或5xx）的响应。可以使用响应对象的`.raise_for_status()`方法来引发该异常。
- **requests.exceptions.HTTPError**

```python
import requests
from requests.exceptions import HTTPError  # 导入 HTTP 错误异常类

# ==============================================
# HTTP 状态码检查机制
# ==============================================
# 背景：requests 默认不会因为 404 或 500 报错，它只负责收发数据。
# 目的：通过 raise_for_status() 强制检查状态码，让“请求失败”变成“程序报错”，便于统一处理。

try:
    # 发送请求
    response = requests.get(url)

    # 【核心代码】
    # 检查响应状态码。如果是错误状态码 (如 404 Not Found)，则抛出异常。
    # 这是一种“防御性编程”的好习惯。
    response.raise_for_status()

    print('响应正常，执行下一步')

except HTTPError as e:
    # ==============================================
    # HTTPError 触发场景详解：
    # ==============================================
    # 1. 400 Bad Request：请求参数错误
    # 2. 401 Unauthorized：未登录/无权限
    # 3. 403 Forbidden：服务器拒绝访问（可能有反爬虫机制）
    # 4. 404 Not Found：请求的页面不存在
    # 5. 500 Internal Server Error：服务器内部代码报错
    # 6. 503 Service Unavailable：服务器过载或维护中

    print("HTTP错误:", str(e))
```

5.3 捕获连接失败异常

- ConnectionError 用于捕获请求连接失败的异常，例如：无法连接到目标服务器，或者网络连接中断。
- **requests.exceptions.ConnectionError**

```python
import requests
from requests.exceptions import ConnectionError  # 导入 requests 库中定义的连接错误异常

# ==============================================
# 异常处理机制
# ==============================================
# 作用：防止因网络问题导致程序直接崩溃
# try 块：包含可能出错的代码
# except 块：定义出错后的处理逻辑

try:
    # 发送 GET 请求
    # 风险提示：这里没有设置 timeout 参数
    # 如果服务器存在但不响应（例如丢包），程序会无限期等待
    response = requests.get(url)
```

```
    # 只有请求成功连接并收到响应，才会执行这里
    print('响应正常，执行下一步')

except ConnectionError as e:
    # ==========================================
    # ConnectionError 触发场景详解：
    # ==========================================
    # 1. DNS 解析失败：网址不存在或域名输入错误 (如 www.googlecom)
    # 2. 拒绝连接：服务器端口未开放，或防火墙拦截 (Connection refused)
    # 3. 网络不可达：本地网络断开，无法连接到互联网
    # 4. 代理连接失败：使用了无效的代理服务器

    print("连接错误:", str(e))  # 打印具体的错误原因
```

5.4 捕获请求超时异常

- `Timeout` 捕获超时异常。
- **requests.exceptions.Timeout**

> 这段代码的核心作用是：**设置请求超时限制，并捕获超时异常以防止程序崩溃。**
>
> 简单来说，它就是给网络请求加了一个“闹钟”。如果请求在规定时间内没完成，就强制停止并报错，而不是让程序一直傻等。
>
> 下面详细解析它的作用和关键点：
>
> **核心作用：防止程序“假死”**
>
> - **不设置超时：** 如果网络波动或服务器无响应，你的程序可能会一直卡在 `requests.get()` 这一行，永远等下去。这在自动化脚本或爬虫中是非常致命的。
> - 设置 `timeout=5`：
>
> 意味着给请求设定了5秒的死线。
>
> - 如果 5 秒内服务器响应了，程序继续往下走。
> - 如果 5 秒到了还没响应，`requests` 会主动抛出 `Timeout` 异常，强制中断这次请求。

```
import requests
from requests.exceptions import Timeout

try:
    response = requests.get(url, timeout=5)
    print('响应正常，执行下一步')

except Timeout as e:
    print("请求超时:", str(e))
```

---

6. 其他方法

6.1 文件上传

语法：

```
requests.post(
    url     #必选参数，指定URL地址
    files   #可选参数，指定文件名
)
```

发送单个文件：

> 这段代码的作用非常明确：**将本地的单个文件上传到指定的服务器。**
>
> 它是使用 Python 的 `requests` 库实现**文件上传（模拟表单提交）**的最标准、最基础的写法。
>
> 下面详细解析它的核心作用和关键细节：
>
> **核心作用：模拟文件上传表单**
>
> - **场景：** 就像你在网页上点击“选择文件”按钮，选中一个文件，然后点击“提交”按钮一样。
> - **协议：** 代码中的 `requests.post(url, files=files)` 会自动将请求头设置为 `Content-Type: multipart/form-data`。这是浏览器上传文件时的标准协议格式。

```python
import requests

url = 'http://xxx.com/'
file_path = './file.txt'

# 读取文件
with open(file_path, 'rb') as file:
    # 将读取的二进制信息赋值为字典
    files = {'file': file}
    # 通过字典发送post请求
    response = requests.post(url, files=files)

# 获取响应信息
print(response.text)
```

发送多个文件：

> 这段代码的核心作用是：**批量上传多个本地文件到服务器，并收集每个文件上传后的响应结果。**
>
> 简单来说，它就是一个**自动化文件上传脚本。**
>
> 下面详细解析它的具体功能和逻辑：
>
> **1. 核心功能：模拟表单文件上传**
>
> - **关键词：** `requests.post(url, files=files)`
> - **作用：** 这行代码模拟了网页上“选择文件并点击提交”的操作。
> - **原理：** 它会将文件内容以 `multipart/form-data`（多部分表单数据）的格式进行编码并发送给服务器。这是 HTTP 协议中传输文件的标准方式。
>
> **2. 执行逻辑：逐个上传**
>
> 代码并没有一次性把所有文件打包上传，而是使用了 `for` 循环：
>
> 1. **遍历字典：** 遍历 `file_paths` 字典，获取表单字段名（`field_name`）和文件路径（`file_path`）。

2. **打开文件：** `with open(file_path, 'rb')` 以**二进制读取模式**打开文件（上传文件必须使用二进制模式）。
3. **发送请求：** 构造 `files` 参数，发送 POST 请求。
4. **记录结果：** 将服务器返回的内容 `response.text` 存入 `uploaded_files` 字典中，以便后续查看。

3. 实际应用场景

假设你要开发一个批量处理系统，这段代码可能用于：

- **批量提交文档：** 比如将本地的 100 个简历文件（PDF/Word）逐一上传到招聘网站的后台。
- **批量图片上传：** 将本地文件夹里的图片逐一上传到图床服务器。
- **数据接口测试：** 测试服务器的文件上传接口是否能正常工作。

```python
import requests

url = 'http://xxx.com/'
file_paths = {
    'file1': './file1.txt',
    'file2': './file2.txt'
}

uploaded_files = {}

# 通过循环方式读取多个文件
for field_name, file_path in file_paths.items():
    with open(file_path, 'rb') as file:
        files = {field_name: file}
        response = requests.post(url, files=files)
        uploaded_files[field_name] = response.text

print(uploaded_files)
```

6.2 代理设置

- 在计算机网络中，代理（Proxy）是一种充当中间人的服务器，它在客户端和目标服务器之间起到转发和代理的作用。当客户端发送请求时，代理服务器接收请求并将其转发给目标服务器。然后，代理服务器接收来自目标服务器的响应，并将其转发回客户端。
- 在使用 requests 库发送 HTTP 请求时，代理（Proxy）可以用于访问被限制或无法直接访问的网站、隐藏真实的 IP 地址、实现访问控制等功能。通过配置代理，请求将通过代理服务器，而不是直接与目标服务器通信。这样，目标服务器会认为请求来自代理服务器，而不是真实的客户端。

1. 通过proxies参数设置全局代理：

1. **核心特点：单次生效（一次性）**
   - **作用范围：** 仅对这一行代码中的 `requests.get(url, proxies=proxies)` 生效。
   - **后果：** 如果你紧接着写下第二行 `requests.get(url2)`（没有带 proxies 参数），这第二个请求**不会走代理**，而是直连。
   - 对比：
     - **环境变量法：** 设置一次，全局生效。
     - **Session法：** 设置一次，会话内生效。
     - **本段代码：** 用一次，生效一次。

2. **优缺点分析**

优点：

- **精准控制：** 在复杂的爬虫逻辑中，有时候我们希望请求 A 走代理 IP1，请求 B 走代理 IP2，请求 C 直连。只有这种方式能灵活实现这种“混合策略”。
- **直观：** 代码读起来非常清晰，一眼就能看出这个请求走了代理。

缺点：

- **代码冗余：** 如果你要发 100 个请求，且都要走同一个代理，你需要写 100 遍 `proxies=proxies`，比较麻烦。
- **维护成本：** 如果要修改代理地址，需要修改 100 处（除非你像代码里那样用一个变量 `proxies` 提前定义好）。

```
import requests

# 设置服务器地址和端口
proxies = {
    'http': 'http://xxx.com:8080',
    'https': 'https://xxx.com:8080'
}

# 通过proxies发送
response = requests.get(url, proxies=proxies)
```

2. 使用环境变量设置代理：

这段代码的核心作用是：**通过设置操作系统环境变量，实现“全局代理”效果。**

简单来说，它的作用是**“一劳永逸”**——在程序开头设置一次，后续所有的请求都会自动走代理，而不需要在每个请求函数里单独写 `proxies` 参数。

下面详细解析它的独特之处：

1. 核心作用：全局配置代理

- **之后的代码：** 是**“局部代理”**。你必须显式地把 `proxies` 参数传给 `requests.get()`。如果你代码里有 100 个请求，你就得写 100 次 `proxies=...`。
- 这段代码：是“全局代理”

  。它把代理配置写进了当前程序运行的环境变量里。

  - `requests` 库的设计机制是：如果用户没有手动指定 `proxies` 参数，它会自动去环境变量里找。
  - **效果：** 代码里后续所有的 `requests.get()`、`requests.post()` 甚至第三方库基于 requests 发出的请求，都会自动检测到并使用这个代理。

2. 代码简洁性（解耦）

- 优势：

  这种方式将“代理配置”和“业务逻辑”分离开了。

  - 你的业务代码可以写得很干净：`requests.get(url)`，不需要在后面拖着一个长长的参数尾巴。
  - 这对于修改和维护非常方便。如果哪天不想用代理了，只需要注释掉开头的两行环境变量设置即可，不需要去满代码里找哪里写了 `proxies`。

```python
import requests
import os

# 通过os获取本地变量去设置代理
os.environ['HTTP_PROXY'] = 'http://xxx.com:8080'
os.environ['HTTPS_PROXY'] = 'https://xxx.com:8080'

# 发送请求时，自动在环境变量中读取代理设置
response = requests.get(url)
```

3. 环境变量中读取代理设置：

> 这段代码的核心作用依然是**通过代理服务器发送请求**，但它引入了一个新的概念：**会话。**
>
> 简单来说，它的作用是：**建立一个持久的、通过代理传输的会话连接，以便在多次请求中保持某些状态（如 Cookie）。**
>
> 下面详细解释它与前一段代码的区别和独特作用：
>
> **1. 核心作用：保持会话状态**
>
> 这是这段代码最大的亮点。
>
> - **后一段代码：** 每次调用 `requests.get()` 都是一次独立的请求。如果你先登录了网站，再请求另一个页面，requests 默认不会记住你登录的状态。
> - 这一段代码： 使用了
>
> ```
> requests.Session()
> ```
>
> - 它就像打开了一个真实的浏览器窗口。
> - 它会**自动处理和持久化 Cookie。**
> - **场景示例：** 很多网站需要先访问首页获取 Cookie，或者需要登录后才能访问数据。使用 Session 可以让你先请求登录接口，Session 会自动保存登录后的 Cookie，后续的请求就能自动带上这个 Cookie，从而保持登录状态。
>
> 2. **代理设置方式的区别**
>
> - **前一段代码：** 将代理作为参数 `proxies=...` 传递给 `requests.get()`。这种方式适合单次请求。
> - **这一段代码：** 将代理设置给 `session.proxies` 对象。这意味着，在这个 Session 生命周期内发出的**所有请求**（无论是 get、post 还是 put），都会默认走这个代理，不需要每次都重复写代理参数。代码更简洁，配置更集中。
>
> 3. **性能提升**
>
> - **连接复用：** `requests.Session()` 底层使用了 `urllib3` 的连接池。
> - **作用：** 如果你连续向同一个网站发送多个请求（比如爬取100个页面），Session 会复用底层的 TCP 连接（Keep-Alive），而不需要每次都重新建立三次握手连接。这能显著提高爬虫的运行速度。

```python
import requests

# 定义代理的地址和端口
proxy = 'http://xxx.com:8080'

# 创建一个会话
session = requests.Session()

# 设置代理
session.proxies = {'http': proxy, 'https': proxy}

# 发送请求
response = session.get(url)
```

4. 代理中设置用户、密码：

> 这段代码的核心作用是：**通过代理服务器发送 HTTP 请求。**
>
> 简单来说，就是让你的程序“伪装”成另一个 IP 地址去访问网站，而不是用你自己电脑真实的 IP 地址。
>
> 具体可以分为以下几个详细作用：
>
> 1. **隐藏真实 IP 地址（保护隐私）**
>
> - **不使用代理：** 目标网站的服务器会记录下你电脑的真实 IP 地址。
> - **使用代理：** 目标网站看到的是代理服务器的 IP 地址，而不是你的真实 IP。这可以保护你的隐私和匿名性。
>
> 2. **突破 IP 封锁限制（爬虫必备）**
>
> - **场景：** 很多网站（如淘宝、知乎、谷歌）有反爬虫机制。如果你在短时间内发送了大量请求，网站会识别出你的 IP 异常并将其封禁（加入黑名单）。
> - **作用：** 使用代理后，如果这个 IP 被封了，你可以切换代码里的代理 IP（换成 `xxx.com:8081` 等），继续访问网站。这是网络爬虫能够持续运行的关键技术。
>
> **3. 突破访问限制（地理限制）**
>
> - **场景：** 某些网站只允许特定国家或地区的 IP 访问（例如某些视频网站限制海外访问，或者某些企业内网系统）。
> - **作用：** 如果你使用的是该地区的代理服务器，你就可以“伪装”成当地用户，从而访问这些受限资源。
>
> **4. 身份认证（代码中的关键点）**
>
> - **细节：** 你提供的代码中包含 `username:password`。
> - **作用：** 这说明使用的是**付费/私密代理**。市面上免费的代理通常不需要账号密码，但很不稳定。付费代理为了保证质量，要求用户在连接时进行身份验证，确保只有付费用户才能使用该代理服务。

```python
import requests

proxies = {
    # 设置用户 : 密码 @ 地址 : 端口
    'http': 'http://username:password@xxx.com:8080',
    'https': 'https://username:password@xxx.com:8080'
}

# 通过设置的代理，发送请求
response = requests.get(url, proxies=proxies)
```

6.3 快速读取多个地址信息

```python
import requests  # 导入 requests 库，用于发送 HTTP 请求

def get_html(url):
    """
    获取指定 URL 的 HTML 内容

    参数:
        url (str): 要请求的网址

    返回值:
        str: 如果请求成功（状态码 200），返回网页 HTML 内容
        None: 如果请求失败，返回 None
    """
    response = requests.get(url)  # 发送 GET 请求到指定 URL

    # 检查响应状态码是否为 200（请求成功）
    if response.status_code == 200:
        return response.text  # 返回网页的 HTML 内容
    else:
        return None  # 请求失败，返回 None

def main():
    """
    主函数
    定义要爬取的 URL 列表，并依次获取每个网页的 HTML 内容
    """
    # 定义要爬取的网站 URL 列表
    urls = [
        'https://www.xxx1.com/',
        'https://www.xxx2.com/',
        'https://www.xxx3.com/'
    ]

    # 遍历 URL 列表，依次获取每个网页的 HTML
    for url in urls:
        html = get_html(url)  # 调用 get_html 函数获取网页内容
        print(html)  # 打印获取到的 HTML 内容

# 判断是否直接运行该脚本（而不是作为模块导入）
if __name__ == '__main__':
    main()  # 执行主函数
```

补充

您提供的图片内容是关于 **Requests 接口自动化测试** 中的“接口关联”技术。这在自动化测试中非常关键，尤其是在测试一个业务流程时，比如：先登录获取 `token`，然后用这个 `token` 去调用其他需要认证的接口。

我们来详细解释一下：

一、什么是“接口关联”？

在接口自动化测试中，“接口关联”指的是**将前一个接口的响应结果（如返回的 token、session id、用户 ID 等）提取出来，并作为参数传递给后续的接口请求。**

例如：

1. **登录接口** 返回了 `{ "token": "abc123" }`
2. **查询订单接口** 需要传入 `Authorization: Bearer abc123`

这时，就需要从登录接口的响应中提取 `abc123` 这个值，然后把它填到查询订单接口的请求头里。这个过程就是“接口关联”。

一、接口关联的三种实现层次

1. 通过类变量保存中间变量

这是最常见和推荐的方式之一。

```
import requests

class TestInterface:
    # 定义一个类变量来存储全局状态
    token = None

    def test_login(self):
        url = "http://api.example.com/login"
        data = {"username": "test", "password": "123456"}

        response = requests.post(url, data=data)
        # 从响应中提取 token 并存入类变量
        self.token = response.json()["token"]

    def test_query_order(self):
        url = "http://api.example.com/orders"
        headers = {
            "Authorization": f"Bearer {self.token}"  # 使用之前存的 token
        }

        response = requests.get(url, headers=headers)
        # 断言...
```

优点: 代码结构清晰，易于维护。
缺点: 如果测试用例之间有依赖，需要注意执行顺序。

2. 通过单独的文件保存中间变量

当测试用例比较复杂或跨多个文件/模块时，可以使用文件（如 `.json`，`.txt`）来存储中间变量。

```
# 写入 token 到文件
with open("token.txt", "w") as f:
    f.write("abc123")

# 读取 token
with open("token.txt", "r") as f:
    token = f.read()
```

优点: 解耦性强，不同脚本可以共享数据。
缺点: 操作文件增加了复杂性，可能受文件权限影响。

3. 极限封装成零代码的方式实现接口关联

这通常指使用一些高级框架（如 `pytest` + `conftest.py` 或专门的自动化测试工具），将提取和传递逻辑完全封装起来，让测试用例本身看起来像“零代码”，即不需要手动写提取和赋值的代码。

优点: 测试用例简洁，维护成本低。
缺点: 学习成本高，灵活性可能受限。

二、接口关联的两种方式：正则表达式提取

这是从接口响应文本中提取所需值的核心技术。

(1) `re.search()` 提取单个值

- **功能:** 在字符串中搜索第一个匹配正则表达式的子串。
- **返回值:** 一个 `Match` 对象，如果没找到则返回 `None`。
- **取值:** 使用 `.group(1)` 或 `[1]` 来获取捕获组中的值。

```
import re

# 假设这是从接口收到的一个响应文本（JSON格式的字符串）
response_text = '{"token":"abc123","user_id":123}'

# 定义一个正则表达式规则，用来匹配 token 的值
# r'...' 表示原始字符串，防止反斜杠被转义
# '"token":"' - 匹配字面量的 "token":
# '(' - 开始一个捕获组，我们想把括号里的内容提取出来
# '[^"]+' - 匹配一个或多个非双引号的字符，这正好是我们想要的 token 值
# ')' - 结束捕获组
# '"' - 匹配结尾的双引号
pattern = r'"token":"([^"]+)"'

# 使用 re.search() 在 response_text 中寻找第一个符合 pattern 规则的内容
# 如果找到了，match 会是一个 Match 对象；如果没找到，match 会是 None
match = re.search(pattern, response_text)

# 判断是否成功匹配到了内容
if match:
    # 从 Match 对象中提取捕获组里的内容
    # group(1) 或 [1] 表示提取第一个括号 () 里的内容
    token = match.group(1)
    print(f"成功提取到 token: {token}")  # 输出: 成功提取到 token: abc123
else:
```

```
    print("未找到 token")  # 如果匹配失败，则打印此信息
```

(2) `re.findall()` 提取多个值

- **功能:** 在字符串中搜索所有匹配正则表达式的子串。
- **返回值:** 一个包含所有匹配项的 **列表 (List)**。
- **取值:** 直接通过下标访问列表元素。

```
import re

# 假设这是从接口收到的响应，包含了一个用户列表
response_text = '{"users":[{"id":1,"name":"Alice"},{"id":2,"name":"Bob"}]}'

# 定义一个正则表达式规则，用来匹配所有的 name 字段
# 与 search 类似，'(' 和 ')' 定义了我们要提取的部分
# '"name":"' 匹配 "name":
# '([^"]+)' 匹配名字内容
# '"' 匹配结尾的双引号
pattern = r'"name":"([^"]+)"'

# 使用 re.findall() 在 response_text 中寻找所有符合 pattern 规则的内容
# 这个方法会返回一个列表 (list)，里面包含了所有匹配到的值
names = re.findall(pattern, response_text)

# 打印整个列表
print(f"提取到的所有名字: {names}")  # 输出: 提取到的所有名字: ['Alice', 'Bob']

# 可以通过索引访问列表中的每一个值
print(f"第一个用户的名字: {names[0]}")  # 输出: 第一个用户的名字: Alice
print(f"第二个用户的名字: {names[1]}")  # 输出: 第二个用户的名字: Bob
```

总结:

- `re.search()`：适合提取**单个**、唯一的值（如 token）。
- `re.findall()`：适合提取**多个**、重复出现的值（如列表中的所有用户名）。

三、JsonPath使用

要理解**JsonPath**的用法，我们可以通过一个具体的JSON数据示例，展示如何使用`jsonpath.jsonpath()`提取信息。JsonPath是一种类似XPath的查询语言，用于从JSON结构中定位和提取数据，其核心语法包括：

- `$`：表示JSON的根节点
- `.`：访问子节点（如`$.store.book`表示根节点下的`store`对象的`book`属性）
- `[*]`：匹配数组中的所有元素（如`$.store.book[*]`表示`book`数组中的所有对象）
- `?()`：过滤条件（如`[?(@.author=='张三')]`表示筛选`author`为"张三"的元素）

示例：提取JSON中的特定字段

假设我们有以下JSON数据（模拟一个包含书籍和自行车的商店信息）：

```
{
  "store": {
    "book": [
      {
```

```
        "title": "JSONPath教程",
        "author": "张三",
        "price": 29.99
      },
      {
        "title": "Python入门",
        "author": "李四",
        "price": 39.99
      }
    ],
    "bicycle": {
      "color": "红色",
      "price": 199.99
    }
  }
}
```

步骤1：导入`jsonpath`库并准备数据

```
import jsonpath  # 需先安装：pip install jsonpath
data = {
  "store": {
    "book": [
      {"title": "JSONPath教程", "author": "张三", "price": 29.99},
      {"title": "Python入门", "author": "李四", "price": 39.99}
    ],
    "bicycle": {"color": "红色", "price": 199.99}
  }
}
```

步骤2：提取所有书的标题

使用JsonPath表达式`$.store.book[*].title`（表示：从根节点开始，找到`store`下的`book`数组，遍历所有元素，提取`title`字段）：

```
titles = jsonpath.jsonpath(data, '$.store.book[*].title')
print(titles)  # 输出：['JSONPath教程', 'Python入门']
```

步骤3：提取作者为"张三"的书标题

使用过滤条件`[?(@.author=='张三')]`（表示：筛选`author`等于"张三"的元素）：

```
zhangsan_books = jsonpath.jsonpath(data, '$.store.book[?(@.author=="张三")].title')
print(zhangsan_books)  # 输出：['JSONPath教程']
```

步骤4：提取不存在的字段（返回None）

如果尝试提取JSON中不存在的字段（如`publisher`），`jsonpath.jsonpath()`会返回`None`：

```
publisher = jsonpath.jsonpath(data, '$.store.book[*].publisher')
print(publisher)  # 输出：None
```

关键说明

- `jsonpath.jsonpath(data, expr)`的返回值：
    - 若找到匹配数据，返回**列表**（即使只有1个匹配项，也用列表包裹）；
    - 若未找到匹配项，返回`None`（而非空列表）。
- 语法灵活性：JsonPath支持复杂的查询（如数组索引、过滤、多级嵌套等），适用于从复杂JSON中精准提取数据。
  通过这个例子，你可以看到JsonPath如何通过简洁的表达式，快速定位JSON中的目标数据，且未找到时明确返回`None`，便于代码判断。